SOFTWARE ARCHITECTURE

Applied Assessment: Layered Architecture

Total: 100 marks | Time: 2 hours | Open-resource (open-internet permitted)

Instructions for Students

- This assessment is open-resource: you may consult notes, textbooks, and online tools, including AI assistants.
- However, every question is built around a specific scenario, set of numbers, or code snippet given below. Marks are awarded for answers that correctly and precisely engage with the SPECIFIC details given — generic, textbook-style answers that do not reference the given numbers, module names, or code will receive little or no credit.
- Where a diagram is requested, you may draw it by hand and submit a photo, or use any diagramming tool. Diagrams must be clearly labelled.
- Show all working for any calculation question. An unexplained final number receives partial credit only.
- Answers should be concise. Suggested word limits are given in square brackets where relevant — exceeding them will not earn extra marks.

Question 1: Designing a Layered Architecture for MediTrack (20 marks)

MediTrack is a pharmacy and inventory management system used across a network of 6 private hospital branches in Sri Lanka. It manages drug inventory, prescription dispensing, and billing data. The system must satisfy the following requirements:

Requirement 1 — Offline-first dispensing: Branches frequently lose internet connectivity. Pharmacists must be able to dispense medication and update local stock counts while offline. Data must sync to the central server automatically once connectivity returns, with conflicts resolved by the central server.

Requirement 2 — Role-based access: Three roles use the system — Pharmacist (dispense medication, view/update stock), Nurse (request medication, view patient prescriptions, no dispensing rights), and Auditor (read-only access to all historical records, cannot create or modify anything).

Requirement 3 — Immutable audit log: Every dispensing action and every stock adjustment, by any role, must be written to an audit log that cannot be edited or deleted by anyone, including system administrators, and must support queries by date range and branch.

Requirement 4 — Controlled-substance reporting: For Schedule II drugs (e.g. morphine, pethidine), each dispensing transaction must be reported within 24 hours to the National Medicines Regulatory Authority (NMRA) via its SOAP API. This external API is slow (4–8 second average response time) and is sometimes unavailable for several hours at a time.

Requirement 5 — Legacy billing integration: After a successful dispensing transaction, MediTrack must send cost data to the hospital's existing Billing System (a separate, SOAP-based legacy platform, contractually locked in for at least 2 more years). MediTrack must never read from the Billing System during a dispensing transaction, and a billing failure must never block or roll back a dispensing transaction.

Tasks

a) **[8 marks]** Draw a layered architecture diagram for MediTrack with a minimum of four layers. Clearly label each layer and draw arrows to show every permitted communication path between layers and external systems (NMRA, Billing System).

b) **[4 marks]** For each of Requirements 1–5, state which layer(s) in your diagram are primarily responsible for satisfying it. One sentence of justification per requirement is sufficient. [≤120 words total]

c) **[4 marks]** Requirements 1 (offline sync) and 3 (audit logging) are cross-cutting concerns — every layer that performs a write operation needs to trigger them. Explain how each could be implemented WITHOUT every layer containing hard-coded calls to the audit/sync mechanism. Name one specific design pattern for each (a different pattern for each of the two concerns).

d) **[4 marks]** Explain, with reference to the NMRA API's response time and availability, why this integration (Requirement 4) should NOT be called synchronously as part of the same request/transaction that performs the local dispensing in Requirement 1. State precisely where in your diagram this integration sits and describe the mechanism by which the dispensing flow hands off to it.

Question 2: Identifying Layering Violations in Code (15 marks)

The following TypeScript-like pseudocode implements an order-creation endpoint for an e-commerce backend. Read it carefully — it contains multiple distinct layering and architectural problems.

```
class OrderController {
  async createOrder(req, res) {
    const userId = req.session.userId;
    const items = req.body.items;

    // 1. check stock directly from DB
    const db = new PostgresConnection(DB_CONFIG);
    for (const item of items) {
      const result = await db.query(
        `SELECT quantity FROM inventory WHERE product_id = ${item.productId}`
      );
      if (result.rows[0].quantity < item.qty) {
        return res.status(400).send("Insufficient stock");
      }
    }

    // 2. calculate total
    let total = 0;
    for (const item of items) {
      const priceRow = await db.query(
        `SELECT price FROM products WHERE id = ${item.productId}`
      );
      total += priceRow.rows[0].price * item.qty;
    }

    // 3. apply loyalty discount - business rule embedded here
    const user = await db.query(
      `SELECT loyalty_tier FROM users WHERE id = ${userId}`
    );
    if (user.rows[0].loyalty_tier === 'gold') {
```

```
      total = total * 0.9;
    }

    // 4. charge payment directly
    const stripeResponse = await stripe.charges.create({
      amount: total * 100,
      currency: 'lkr',
      source: req.body.cardToken
    });

    // 5. insert order, return raw row to frontend
    const order = await db.query(
      `INSERT INTO orders (user_id, total, stripe_id) VALUES (${userId}, ${total},
'${stripeResponse.id}') RETURNING *`
    );

    res.json(order.rows[0]);
  }
}
```

Tasks

a) **[8 marks]** Identify FIVE distinct layering/architecture violations in this code (the numbered comments are hints, but you must find five separate issues, not five line numbers — some comments may correspond to more than one issue). For each violation: (i) name the specific principle or concept it violates (e.g. 'Separation of Concerns', 'Dependency Inversion Principle', 'leaky abstraction' — not just 'bad practice'), and (ii) state which layer this responsibility SHOULD belong to in a properly layered design.

b) **[3 marks]** One of the issues you identified in (a) is also a serious SECURITY vulnerability, independent of layering concerns. Identify which one, name the vulnerability class, and explain concretely how an attacker could exploit it.

c) **[4 marks]** Redesign this flow using proper layering. List the classes/interfaces you would introduce across the Presentation, Application/Service, Domain, and Persistence layers (one or two per layer is enough), and show — as a short numbered call sequence (not full code) — how a call to create an order would flow through them, including where the loyalty discount rule and the payment call would now live.

Question 3: The Sinkhole Anti-Pattern at EduPortal (15 marks)

EduPortal is a strict, closed 5-layer system: Presentation (P) → Application (A) → Domain (D) → Persistence (Pe) → Database (DB). Every call must pass through each adjacent layer in order; no layer may be skipped.

An architecture audit examined the 12 read-only (GET) endpoints in the system and found the following:

- For 8 of the 12 endpoints, the Application-layer method does nothing except invoke the corresponding Domain-layer method with the same parameters and return its result unchanged — no validation, transformation, aggregation, or authorization logic of any kind.
- Of those same 8 endpoints, 5 also have a Domain-layer method that does nothing except invoke Persistence and return the result unchanged.

- The remaining 4 endpoints have genuine logic in both their Application and Domain layers (e.g. combining data from multiple Persistence calls, applying business rules, or enforcing access checks).

Tasks

a) **[3 marks]** Define the 'sinkhole anti-pattern' in one or two sentences. Then state, with exact counts, which layers in EduPortal qualify as sinkholes, for how many endpoints, and which endpoints (by number, e.g. '4 of the 12') do NOT exhibit this pattern at all.

b) **[4 marks]** Across the Application and Domain layers only, there are 24 (endpoint × layer) instances in total (12 endpoints × 2 layers). Calculate what percentage of these 24 instances are 'pure pass-through' (sinkhole) instances. Show your working step by step.

c) **[5 marks]** Propose a refactor that removes unnecessary indirection ONLY for the affected endpoints, while preserving the full 5-layer call path for the 4 endpoints with real logic. Describe (in words, or with a small diagram) the resulting call path for: (i) one of the 5 endpoints where BOTH Application and Domain are sinkholes, (ii) one of the 3 endpoints where only Application is a sinkhole, and (iii) one of the 4 endpoints with real logic.

d) **[3 marks]** Is it architecturally acceptable for a single system to have different call-path lengths for different endpoints, as your refactor in (c) produces? Discuss the trade-off between consistency/predictability of the architecture versus the cost of unnecessary indirection, and give your recommendation for EduPortal specifically.

Question 4: Quantifying the Cost of Strict Layering at StockWatch (15 marks)

StockWatch is a financial dashboard. Measured average one-way overhead added by each layer transition (independent of the actual work done) is:

Layer transition	Overhead (round trip)
Presentation → Application (P→A)	1.5 ms
Application → Domain (A→D)	1.0 ms
Domain → Persistence (D→Pe)	2.0 ms
Persistence → Database (Pe→DB)	4.0 ms

On page load, the 'Top Movers' widget issues 20 independent calls to a read-only `getStockPrice(symbol)` operation, which under strict layering travels P→A→D→Pe→DB and back.

A proposed optimisation: since `getStockPrice` 'just looks up a number', allow the Presentation layer to call the Persistence layer directly for this one operation, skipping Application and Domain entirely (relaxed layering).

Tasks

a) **[4 marks]** Calculate the TOTAL layer-transition overhead (excluding actual database execution time) for loading the Top Movers widget (20 calls) under (i) strict layering, and (ii) the proposed relaxed layering. Show the formula you use for each and the final totals in milliseconds.

b) **[3 marks]** Express the saving from (a) as a percentage reduction in layer-transition overhead.

c) **[5 marks]** The Domain layer at StockWatch currently enforces this rule: 'If a stock symbol is on the current user's personal compliance restricted/blacklist (set to prevent insider-trading conflicts), `getStockPrice` must return "N/A" instead of the real price, regardless of what Persistence returns.' If Presentation calls Persistence directly as proposed, explain SPECIFICALLY what breaks for a user with a restricted symbol on their Top Movers list, and why a regulator or auditor reviewing this system would consider it a serious finding — not just an architectural style issue.

d) **[3 marks]** Propose a solution that captures most of the latency benefit calculated in (a) WITHOUT allowing Presentation to bypass the Domain layer's restriction check. Name the specific technique or pattern you would use.

Question 5: Refactoring CarbonMetrics: From Tangled Modules to Layers (20 marks)

CarbonMetrics is a platform used by corporate sustainability teams to log activity data (fuel use, electricity consumption, business travel) and calculate greenhouse-gas (Scope 1/2/3) emissions following the GHG Protocol. The current codebase consists of six modules:

- IngestionAPI — REST endpoints that receive activity-data uploads (CSV files and manual entry forms).
- EmissionFactorDB — reference tables of emission factors (e.g. kg CO_2e per litre of diesel), updated quarterly from external standards bodies.
- CalculationEngine — applies emission factors to activity data to compute CO_2e totals.
- ReportGenerator — produces PDF/Excel sustainability reports for customers.
- UserDashboard — web UI showing emissions trends over time.
- DataStore — a single PostgreSQL database holding all activity data, emission factors, and computed results.

An architecture review found the following dependencies as they currently exist in the codebase:

- IngestionAPI writes activity data directly into DataStore AND directly invokes CalculationEngine for each upload.
- CalculationEngine reads emission factors from EmissionFactorDB by running SQL joins directly against DataStore (there is no API boundary between them).
- ReportGenerator queries DataStore directly with its own SQL and its OWN, separately-written implementation of the emission formula — it does not call CalculationEngine, and its formula has drifted slightly out of sync with CalculationEngine's over time.
- UserDashboard calls ReportGenerator's internal functions directly (in-process function calls), bypassing any API, because 'they're in the same codebase'.
- Quarterly updates to EmissionFactorDB are applied as in-place UPDATE statements on the existing rows in DataStore — there is no versioning or history of which factor value was in effect on any given date.

Tasks

a) **[5 marks]** Draw the CURRENT architecture as a layer/module diagram. On the diagram, identify and label at least THREE specific dependencies listed above that violate clean layered architecture principles, and name the principle each one violates.

b) **[4 marks]** The duplicated emission-calculation logic in ReportGenerator (separate from CalculationEngine) is a serious problem beyond 'poor layering'. Explain the concrete business risk this creates for a CarbonMetrics customer that submits these reports for regulatory disclosure or to an external auditor.

c) **[4 marks]** The lack of versioning for EmissionFactorDB is also a correctness bug, not merely an architecture smell. Construct a concrete example (with example dates and factor values of your choosing) showing how this produces an INCORRECT historical report, and explain how proper versioning, placed in the appropriate layer(s) of a redesigned architecture, would prevent it.

d) **[7 marks]** Propose a target layered architecture for CarbonMetrics with a minimum of five layers. For each of the six existing modules, state which layer(s) it would belong to in your new design (a module may need to be split across layers). Specifically explain how ReportGenerator and CalculationEngine should relate to each other so the duplicated formula in (b) is eliminated, and where/how emission-factor versioning from (c) fits into your design.

Question 6: Open vs. Closed Layering in a Project Management Tool (15 marks)

A project-management web application has the following architecture. Layers, top to bottom: UI (web and mobile clients) → API Gateway → Application Services → Domain/Business Logic → Data Access → Database. Two additional components sit alongside the main stack: a Caching Layer (positioned between Application Services and Data Access) and a Notification Layer (receives events from the Domain layer and forwards them to external email/push-notification services).

The system's communication paths (arrows) are as follows:

- 1. UI → API Gateway
- 2. API Gateway → Application Services
- 3. Application Services → Domain
- 4. Application Services → Caching Layer (for read-heavy endpoints)
- 5. Caching Layer → Data Access (on a cache miss)
- 6. Domain → Data Access
- 7. Domain → Notification Layer (fires events on state changes)
- 8. Data Access → Database
- 9. UI → Caching Layer (used ONLY by the 'live activity feed' widget, for low latency)

Tasks

a) **[3 marks]** For EACH of the 9 arrows listed above, classify it as one of: (i) consistent with strict CLOSED layering (adjacent layers only), (ii) consistent with relaxed/OPEN layering (skips one or more layers but still flows in a permitted direction), or (iii) a violation regardless of open/closed (e.g. an upward call, or a call that bypasses encapsulation in a way no

layering style permits). Present your answer as a simple list (arrow number → classification, one short justifying phrase each).

b) **[4 marks]** Arrow 9 (UI → Caching Layer for the live activity feed) is the most architecturally risky. Identify TWO distinct, specific problems this creates: one related to SECURITY/AUTHORIZATION, and one related to DATA CONSISTENCY/STALENESS. For each, explain what the bypassed layer(s) would normally have provided that is now missing.

c) **[4 marks]** Considered as a whole, is this system best described as ‘open layered’ or ‘closed layered’ architecture? Justify your answer with reference to at least THREE of the specific arrows from part (a). Then describe ONE concrete governance or documentation practice a team should adopt to prevent an open layered system like this from gradually degrading into an unmanageable dependency mesh.

d) **[4 marks]** Propose an alternative design for the ‘live activity feed’ requirement that achieves comparably low latency WITHOUT the UI bypassing the layers as drastically as arrow 9 does. Name a SPECIFIC architectural pattern or technology suited to real-time updates in layered web systems, and briefly describe how data would flow from the Domain layer to the UI in your alternative.